

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  Example 5.4 Softmax decision boundary for the Iris dataset
5  Machine Learning for Engineering Problem Solving
6  @author: Austin R.J. Downey
7  """
8
9  import IPython as IP
10 IP.get_ipython().magic('reset -sf')
11
12 import numpy as np
13 import matplotlib.pyplot as plt
14 import sklearn as sk
15
16
17 cc = plt.rcParams['axes.prop_cycle'].by_key()['color']
18 plt.close('all')
19
20
21 %% Load your data
22
23 # We will use the Iris data set. This dataset was created by biologist Ronald
24 # Fisher in his 1936 paper "The use of multiple measurements in taxonomic
25 # problems" as an example of linear discriminant analysis
26 iris = sk.datasets.load_iris()
27
28 # for simplicity, extract some of the data sets
29 X = iris['data'] # this contains the length of the petals and sepals
30 Y = iris['target'] # contains what type of flower it is
31 Y_names = iris['target_names'] # contains the name that aligns with the type of the
32 # flower
33 feature_names = iris['feature_names'] # the names of the features
34
35 # plot the Sepal data
36 plt.figure(figsize=(6.5,3))
37 plt.subplot(121)
38 plt.grid(True)
39 plt.scatter(X[Y==0,0],X[Y==0,1],marker='o',zorder=10)
40 plt.scatter(X[Y==1,0],X[Y==1,1],marker='s',zorder=10)
41 plt.scatter(X[Y==2,0],X[Y==2,1],marker='d',zorder=10)
42 plt.xlabel(feature_names[0])
43 plt.ylabel(feature_names[1])
44
45 plt.subplot(122)
46 plt.grid(True)
47 plt.scatter(X[Y==0,2],X[Y==0,3],marker='o',label=Y_names[0],zorder=10)
48 plt.scatter(X[Y==1,2],X[Y==1,3],marker='s',label=Y_names[1],zorder=10)
49 plt.scatter(X[Y==2,2],X[Y==2,3],marker='d',label=Y_names[2],zorder=10)
50 plt.xlabel(feature_names[2])
51 plt.ylabel(feature_names[3])
52 plt.legend(framealpha=1)
53 plt.tight_layout()
54
55 %% Softmax Regression
56
57 # build the training and target set.
58 X_train = X[:, (2, 3)] # petal length, petal width
59 y_train = Y
60
61 # build and train the softmax model
62 softmax_reg = sk.linear_model.LogisticRegression(multi_class="multinomial",
63 solver="lbfgs", C=10)
64 softmax_reg.fit(X_train, y_train)
65
66 # build the x values for the predictions over the entire "petal space"

```

```

67 x_grid, y_grid = np.meshgrid(
68     np.linspace(0, 7, 500),
69     np.linspace(0, 4, 200),
70 )
71 X_new = np.vstack((x_grid.reshape(-1), y_grid.reshape(-1))).T # build a vector format of
the mesh grid
72
73 # predict on the vectorized format
74 y_predict = softmax_reg.predict(X_new)
75 y_proba = softmax_reg.predict_proba(X_new)
76
77
78 # convert back to meshgrid shape for plotting
79 zz_predict = y_predict.reshape(x_grid.shape)
80 zz_proba = y_proba[:, 1].reshape(x_grid.shape) # the selected column selects the
probability that the data falls within this class.
81
82 # plot the 2D "petal space"
83 plt.figure(figsize=(6.5, 4))
84 plt.scatter(X[Y==0,2],X[Y==0,3],marker='o',label=Y_names[0],zorder=10)
85 plt.scatter(X[Y==1,2],X[Y==1,3],marker='s',label=Y_names[1],zorder=10)
86 plt.scatter(X[Y==2,2],X[Y==2,3],marker='d',label=Y_names[2],zorder=10)
87 plt.contourf(x_grid, y_grid, zz_predict, cmap='Pastel2')
88 contour = plt.contour(x_grid, y_grid, zz_proba, [0.100,0.5,0.900], cmap=plt.cm.brg)
89 plt.clabel(contour, inline=1)
90 plt.xlabel(feature_names[2])
91 plt.ylabel(feature_names[3])
92 plt.legend()
93 plt.tight_layout()

```